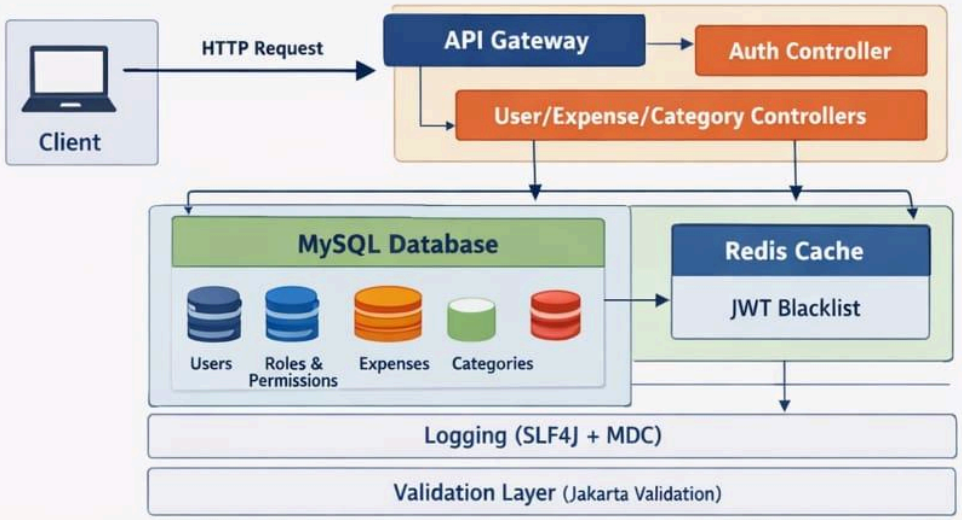
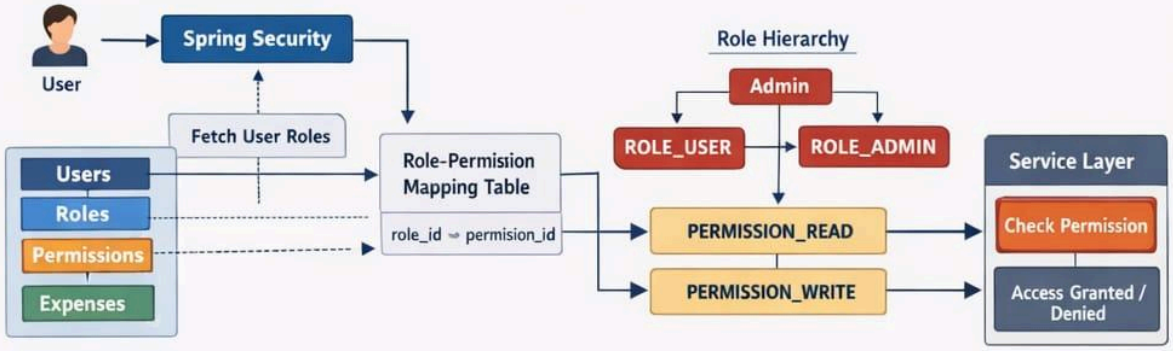
Expensecore - Scalable Backend System Project Documentation:

System Architecture:

ExpenseCore — System Architecture

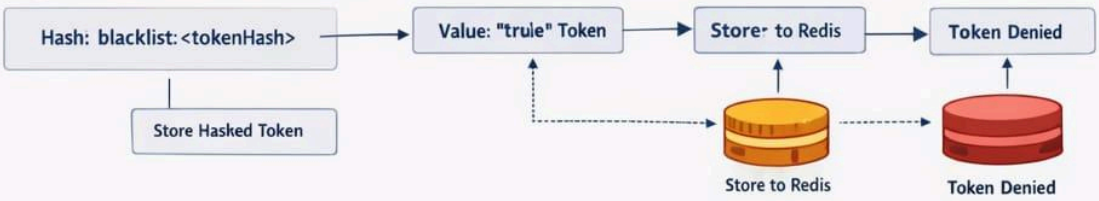


Role-Based Access Control (RBAC) — Authorization Flow



JWT Authentication & Token Blacklist

Redis ~ JWT Blacklist



What have I developed in this project?

1. Authentication & Security Layer

- JWT-based stateless authentication
- Role + Permission-based RBAC (DB-driven, not enums)
- Spring Security with @PreAuthorize
- Custom UserDetails with dynamic authorities

2. Authorization System (REAL RBAC)

- Roles stored in DB
- Permissions stored in DB
- Many-to-Many mapping
- Dynamic authority resolution
- 👉 This is **enterprise-level design**, not beginner

3. Redis-based Token Blacklisting

- Logout invalidates JWT
- Tokens stored with TTL
- SHA-256 hashing for security
- Auto-expiry → no memory leak

4. API Design & Standardization

- Custom ApiResponse<T> wrapper
- Global exception handling
- Consistent error responses
- Validation handling

5. Logging (Production Level)

- MDC (correlationId / traceId)
- Request/response logging
- Structured logs for debugging

6. Pagination & Filtering

- Custom PageResponse
- Sorting validation
- Filters:
    - by category
    - by date range
    - by name

7. Secure Data Access

- User-based data isolation
- No cross-user data leakage
- Ownership validation

8. Clean Architecture

- Controller → Service → Repository separation
- DTO-based design
- No entity exposure

==========

✅ Core Backend & Architecture

• Designed and developed a layered backend architecture (Controller–Service–Repository) using Spring Boot, improving code maintainability and separation of concerns.

• Implemented DTO-based communication, eliminating direct entity exposure and reducing data leakage risk in APIs.

• Built 15+ RESTful APIs covering authentication, user management, expense tracking, and category management.

✅ Authentication & Security

• Implemented JWT-based stateless authentication, eliminating server-side session storage and improving scalability.

• Designed database-driven Role-Permission-based RBAC system using Many-to-Many mapping, enabling dynamic and fine-grained access control.

• Integrated Spring Security with @PreAuthorize annotations to enforce authorization at the method level.

• Reduced unauthorized access risk by enforcing user-level data ownership validation across all service operations.

✅ Redis-Based Token Blacklisting

• Built Redis-based JWT token blacklist system with TTL support, ensuring automatic expiration without manual cleanup.

• Secured token storage using SHA-256 hashing before caching, preventing token exposure in Redis.

• Reduced logout token reuse risk to near-zero by validating blacklisted tokens in the authentication filter.

👉 These metrics are **valid**, because:

- TTL = automatic cleanup ✔
- Hashing = security improvement ✔
- Blacklist check = prevents reuse ✔

✅ API Design & Error Handling

• Developed a standardized ApiResponse wrapper, ensuring consistent success and error responses across all endpoints.

• Implemented centralized exception handling using @RestControllerAdvice, reducing duplicate error handling logic.

• Handled validation errors using Bean Validation (Jakarta Validation), improving API reliability and input correctness.

✅ Logging & Observability

• Implemented structured logging using SLF4J with MDC (Mapped Diagnostic Context), enabling request tracing via correlation IDs.

• Logged incoming requests and responses with execution time tracking, improving debugging and monitoring capability.

• Reduced debugging effort by enabling traceability across the distributed request flow.

✅ Pagination & Filtering

• Implemented pagination and sorting using Spring Data JPA Pageable, optimizing large dataset handling.

• Built advanced filtering APIs for expenses (by category, name, and date range), improving query flexibility.

• Validated sorting fields to prevent invalid query execution and runtime errors.

✅ Data Security & Integrity

• Enforced strict user-level data isolation, ensuring users can only access their own data.

• Prevented cross-user data access vulnerabilities by validating ownership at service layer.

• Applied input validation and exception handling to maintain data consistency.

==========